# A Branch-and-Bound Algorithm to Minimize Total Flow Time with Unequal Release Dates

**Chengbin Chu**
*Projet SAGEP, INRIA-Lorraine, CESCOM, Technopôle Metz 2000*
*4, rue Marconi, 57070 Metz, France*

This article examines the single-machine scheduling problem to minimize total flow time with unequal release dates. This problem has been proven to be NP-hard. We present a necessary and sufficient condition for local optimality which can also be considered as a priority rule. On the basis of this condition, we then define a class of schedules which contains all optimal solutions. We present some efficient heuristic algorithms using the previous condition to build a schedule belonging to this subset. We also prove some new dominance theorems, discuss the results found in the literature for this problem, and propose a branch-and-bound algorithm in which the heuristics are used to provide good upper bounds. We compare this new algorithm with existing algorithms found in the literature. Computational results on problems with up to 100 jobs indicate that the proposed branch-and-bound algorithm is superior to previously published algorithms. 

## 1. INTRODUCTION

The single-machine scheduling problem has been extensively studied under different assumptions and various objective functions. Gupta and Kyparisis [9] reviewed various results related to this problem. In this article we consider the $n$-job, one-machine scheduling problem. Each job of the set $N = \{1, 2, \ldots, n\}$ is to be processed on a single machine which performs at most one job at a time. No preemption is allowed. Each job $i$ becomes available at its release date $r_i$ and requires a processing time $p_i$. If we consider only semiactive schedules (see Baker [2]), then given a processing order of jobs $\sigma$, the flow time $F_i(\sigma)$ for each job $i$ (i.e., the period between the release date $r_i$ and the completion time $C_i(\sigma)$, $F_i(\sigma) = C_i(\sigma) - r_i$) can be computed. When there is no ambiguity, we abbreviate, respectively, $C_i(\sigma)$ and $F_i(\sigma)$ to $C_i$ and $F_i$. We look for a schedule $S$ such that $F(S) = \sum_{i=1}^{n} F_i(S)$ is minimal. This problem is equivalent to the problem of minimizing the total job lateness, total completion time, total waiting time, and mean number of jobs in the shop (Conway, Maxwell, and Miller [6]).

In the case of equal release dates, this problem can easily be solved by applying the SPT (shortest processing time) priority rule (Smith [14]). The SPT priority rule consists of scheduling a job with the shortest processing time among the unscheduled jobs each time the machine becomes idle.

*Naval Research Logistics*, Vol. 39, pp. 859–875 (1992)
 CCC 0894-069X/92/060859-17$04.00

However, it is NP-hard with different release dates (Lenstra, Rinnooy Kan, and Brucker [11], Rinnooy Kan [12]), which indicates that the existence of a polynomially bounded algorithm is unlikely. Several researchers have proven some dominance properties and proposed a number of algorithms. A schedule $S$ dominates another schedule $S'$ if and only if it gives a criterion value at least as good as the one given by $S'$. A dominance property is a property indicating that some schedules dominate some others. A dominant subset is a subset containing at least one optimal schedule. Chandra [4] discussed two models. The first one is the model studied in this article, but without inserted idle time; the second one is with preemptive-repeat. (An operation can be preempted, but the work done before the preemption is lost. If it is processed later, the processing requires a time equal to its initial processing time.) The second model is equivalent to the problem studied in this article. He proposed a branch-and-bound algorithm able to solve problems with up to 30 jobs. Dessouky and Deogun [8] proved several dominance properties for the problem discussed in this paper and proposed a branch-and-bound algorithm which was tested on problems involving 50 jobs. Using a partitioning method, Deogun [7] proposed a branch-and-bound algorithm tested on problems with up to 50 jobs.

For the total weighted completion time criterion, Bianco and Ricciardelli [3] have proven several dominance properties and proposed a branch-and-bound algorithm tested on problems with 10 jobs. Hariri and Potts [10] proposed an approach using the Lagrangian relaxation method to obtain a lower bound in a branch-and-bound enumerative algorithm and succeeded in solving problems with 50 jobs.

In this article we present a necessary and sufficient condition for local optimality. By local optimality, we mean the optimality of two adjacent jobs in a given schedule, without taking into account the jobs following them, assuming that preceding jobs are fixed. This is similar to considering the one-machine two-job problem. This condition can also be considered as a priority rule. Based on this condition, we define a subset, called $F$-active subset, containing all the optimal schedules. Some results in the literature are discussed. We also present some heuristic algorithms to build an $F$-active schedule. Using one of these algorithms to provide good upper bounds and using results presented in this article or found in the literature, we construct a new branch-and-bound algorithm which can solve problems with up to 100 jobs.

In Section 2, we present the necessary and sufficient condition for local optimality and the definition of the $F$-active schedule subset. Section 3 presents the heuristics. Section 4 presents dominance theorems. Section 5 discusses sufficient conditions for optimality. Section 6 presents several lower bounds. Section 7 presents the branch-and-bound algorithm that we propose and the existing ones. The computational results are given in Section 8.

## 2. NECESSARY AND SUFFICIENT CONDITION FOR LOCAL OPTIMALITY, $F$-ACTIVE SCHEDULES

As described in the introduction, obtaining a condition for local optimality is equivalent to considering a two-job scheduling problem. Let us consider this problem. We have to schedule two jobs $i$ and $j$ on a machine available at time

$\Delta$. We are interested in finding a condition whereby we can obtain an optimal schedule. Assuming that $F_{uv}(\Delta)$ is the sum of flow times of jobs $u$ and $v$ obtained by scheduling $u$ before $v$ (in order to simplify the notations, we write $F_{uv}$ when there is no ambiguity), the problem is to find the condition whereby we have $F_{ij} \leq F_{ji}$ when two jobs $i$ and $j$ are to be scheduled.

To simplify the explanation, let us denote $R_i(\Delta) = \max(\Delta, r_i)$ the earliest beginning time of job $i$ at time $\Delta$ and $E_i(\Delta) = R_i(\Delta) + p_i$ the earliest completion time of job $i$ at time $\Delta$. We will show that the condition we are looking for is obtained using a new dynamic priority rule called PRTF (priority rule for total flow time).

DEFINITION 1: We define the function PRTF($i$, $\Delta$) of a job $i$ at the time $\Delta$ as $\text{PRTF}(i, \Delta) = 2 \cdot R_i(\Delta) + p_i = R_i(\Delta) + E_i(\Delta)$.

A necessary and sufficient condition for local optimality can be obtained according to this function.

THEOREM 1: Consider a scheduling problem involving only two jobs $i$ and $j$ which have to be scheduled on a machine available after time $\Delta$. Scheduling $i$ before $j$ is optimal (i.e., the total flow time is minimal) if and only if $\text{PRTF}(i, \Delta) \leq \text{PRTF}(j, \Delta)$. In other words: $\text{PRTF}(i, \Delta) \leq \text{PRTF}(j, \Delta) \Leftrightarrow F_{ij} \leq F_{ji}$.

PROOF: The proof can be found in [5]. ■

According to Theorem 1, we can consider the PRTF function as a dynamic priority rule: The smaller the value of PRTF, the higher the priority of the job. This necessary and sufficient condition enabled us to build a dominant subset to minimize the total flow time. In order to simplify notations, we denote by $\Delta_i$ the completion time of the job immediately preceding job $i$ in a schedule. If $i$ is the first job of the schedule, $\Delta_i = -\infty$.

DEFINITION 2: Consider a given active schedule $S$ (an active schedule is a schedule such that no job can be scheduled earlier without delaying another). If any couple of adjacent jobs $i$ and $j$ ($i$ followed by $j$) verifies at least one of the following conditions: (1) $R_i(\Delta_i) < R_j(\Delta_i)$, (2) $\text{PRTF}(i, \Delta_i) \leq \text{PRTF}(j, \Delta_i)$, $S$ is said to be $F$-active.

According to the previous definition, Theorem 1 leads to the following corollary.

COROLLARY 1: All the optimal solutions are $F$-active.

## 3. HEURISTIC ALGORITHMS

In [5], we constructed two heuristic algorithms using the PRTF function and building an $F$-active schedule. These two heuristics are presented in this section. They will be used to provide good upper bounds in a new branch-and-bound algorithm presented in Section 7. From the computational results presented in [5], we can see that these algorithms are better than traditional priority rules.

All the heuristics definitely schedule a job behind the partial schedule composed of scheduled jobs. At each iteration, the machine begins to be available at time $\Delta$ and there is a set $A$ of unscheduled jobs. The heuristics stop when all the jobs are scheduled; i.e., $A = \emptyset$. $\Delta$ and $A$ are, respectively, initialized to $-\infty$ and $\{1, 2, \ldots, n\}$.

Algorithm PRTF
Select job $i$ with $\min_{i \in A}$ PRTF$(i, \Delta)$. Break ties by choosing $i$ with $\min[R_i(\Delta)]$ and further ties by choosing $i$ with $\min(i)$. Schedule $i$. Update $\Delta$ and $A$ such that $\Delta := E_i(\Delta)$, $A := A - \{i\}$.

Algorithm APRTF
*Step 1*: Select job $\alpha$ with $\min_{\alpha \in A}$ PRTF$(\alpha, \Delta)$. Break ties by choosing $\alpha$ with $\min[R_\alpha(\Delta)]$ and further ties by choosing $\alpha$ with $\min(\alpha)$.
*Step 2*: Select job $\beta$ with $\min_{\beta \in A} R_\beta(\Delta)$. Break ties by choosing $\beta$ with $\min[p_\beta]$ and further ties by choosing $\beta$ with $\min(\beta)$.
*Step 3*: If $r_\alpha \leq R_\beta(\Delta)$, then $x := \alpha$. Go to Step 5.
*Step 4*: Noting $\mu = \text{card}(A - \{\alpha, \beta\})$, $\tau$ the smallest release date of jobs of $A - \{\alpha, \beta\}$, if the condition $F_{\beta\alpha}(\Delta) - F_{\alpha\beta}(\Delta) < \mu \cdot \min[R_\alpha(\Delta) - R_\beta(\Delta), E_\beta(E_\alpha(\Delta)) - \tau]$ holds, then $x := \beta$; otherwise, $x := \alpha$. Go to Step 5.
*Step 5*: Schedule $x$. $\Delta$ is increased to $E_x(\Delta)$, $A := A - \{x\}$.

At step 4, the algorithm chooses a job to schedule by comparing the gain and the loss. If we scheduled $\beta$ at time $\Delta$, there would not be avoidable idle time on the machine. With $\alpha$ scheduled at that time, we respect the priority rule PRTF (see Figure 1).

If we schedule $\alpha$ before $\beta$, we can obtain a gain of $F_{\beta\alpha}(\Delta) - F_{\alpha\beta}(\Delta)$, but we create an idle time of value $R_\alpha(\Delta) - R_\beta(\Delta)$. The unscheduled jobs other than $\alpha$ and $\beta$ are delayed for at most $\min[R_\alpha(\Delta) - R_\beta(\Delta), E_\beta(E_\alpha(\Delta)) - \tau]$. The loss

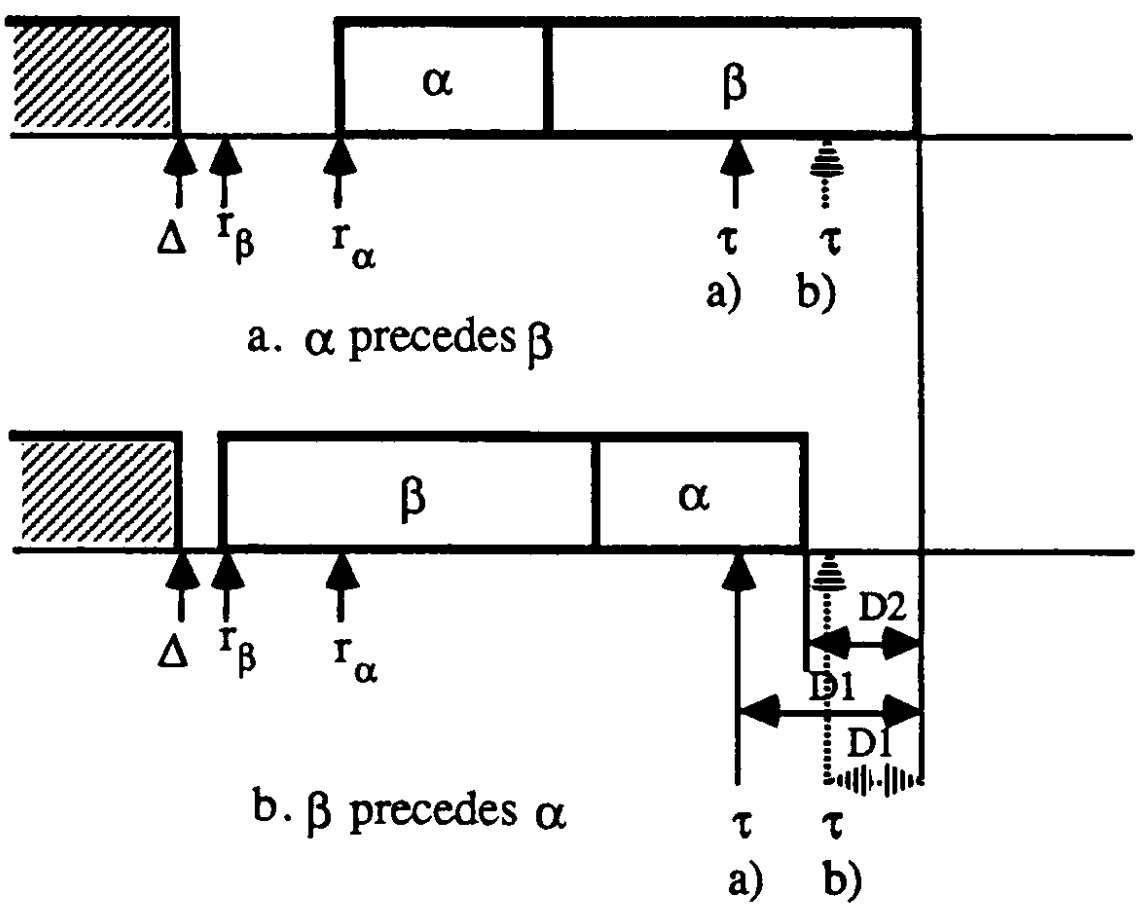


**Figure 1.** Comparison of gain and loss. (a) Case $D1 \geq D2$; (b) Case $D1 < D2$.

in the worst case is $\mu \cdot \min[R_\alpha(\Delta) - R_\beta(\Delta), E_\beta(E_\alpha(\Delta)) - \tau]$. If the gain is at least as great as the loss, we schedule $\alpha$; otherwise $\beta$ is scheduled.

## 4. DOMINANCE PROPERTIES

In this section, we review a number of dominance theorems found in the literature, prove some new ones, and compare these results. This comparison indicates that some dominance theorems are redundant because they are dominated by others. We only use those that are not dominated.

Henceforth, let $K$ denote a partial schedule, $J(K)$ the set of jobs in this partial schedule, and $\Phi(K)$ denote the completion time of the last job in $K$. $K|i$ is the new partial schedule obtained by adding the job $i$ behind the given partial schedule $K$. $\Sigma(K, i)$ is the schedule composed of $K|i$, completed by the partial optimal schedule of jobs belonging to $N - J(K|i)$ starting from the moment $\Phi(K|i)$.

We list in Table 1 some dominance properties indicating the conditions under which the schedule $\Sigma(k, i)$ is dominated. The contribution of this article is to prove some new dominance properties (Theorems 9, 10, and 11) that indicate the conditions under which the schedule $\Sigma(k, i)$ is dominated. We now prove these theorems.

**Table 1.** Dominance properties.

| Theorem | Reference | Conditions | Conclusion |
|---|---|---|---|
| 2 | [8] | $\exists j \in N - J(K)$: $j \neq i$; $p_j \leq p_k$, $\forall k \in N - J(K)$; $R_i(\Phi(K)) \geq R_j(\Phi(K))$ | |
| 3 | [8] | $\exists j \in N - J(K)$: $j \neq i$; $E_j(\Phi(K)) \leq E_k(\Phi(K))$, $\forall k \in N - J(K)$; $r_i \geq E_j(\Phi(K))$ | |
| 4 | [8] | $\exists j \in N - J(K)$: $j \neq i$; $r_i \geq E_j(\Phi(K))$ | |
| 5 | [8] | $\exists j \in N - J(K)$: $j \neq i$; $p_j \geq p_i$; $E_j(\Phi(K)) \leq E_i(\Phi(K))$ | |
| 6 | [3] | $\exists j \in N - J(K)$: $j \neq i$; $E_i(\Phi(K)) \geq E_j(\Phi(K))$; $E_i(\Phi(K)) - E_j(\Phi(K)) \geq (p_i - p_j)[\text{card}(N - J(K)) - 1]$ | |
| 7 | [3] | $\exists j \in N - J(K)$: $j \neq i$; $E_i(\Phi(K)) \leq E_j(\Phi(K))$; $p_i - p_j \leq [E_i(\Phi(K)) - E_j(\Phi(K))] \cdot \text{card}[N - J(K)]$ | $\Sigma(k, i)$ is dominated |
| 8 | [7] | There is a job $j$ in a SPT sequence of $N - J(K)$ starting from the moment $\Phi(K)$ such that $E_j(\Phi(K)) - \Phi(K) \geq \Sigma_{k \in B_j} [R_k(\Delta_k) - \Delta_k]$, $p_j \leq p_i$, and $R_j(\Phi(K)) \leq R_i(\Phi(K))$, where $B_j$ is the set of jobs preceding $j$ in the SPT sequence | |
| 9 | This paper | $\exists j \in J(K)$: $j \neq i$; $E_i(\Delta_j) \leq E_j(\Delta_j)$; $E_i(\Delta_j) - E_j(\Delta_j) \leq (p_i - p_j) \cdot \text{card}(N - J(K))$ | |
| 10 | This paper | $\exists j \in J(K)$ in the $k$th position: $p_i \geq p_j$; $p_i - p_j \geq (E_i(\Delta_j) - E_j(\Delta_j)) \cdot (\text{card}(J(K)) - k + 2)$ | |
| 11 | This paper | There is another partial solution $K'$: $J(K') = J(K) \cup \{i\}$; $F(K') \leq F(K|i)$; $\text{card}(N - J(K')) \cdot R_j(\Phi(K')) + F(K') \leq \text{card}(N - J(K')) \cdot R_j(\Phi(K|i)) + F(K|i)$, where $j$ is a job of $N - J(K')$ with the smallest release date | |

PROOF (of Theorem 9): Consider the schedule $\Sigma(K, i)$. We construct another schedule $S$ by interchanging the positions of jobs $i$ and $j$.

In the case where $p_i \geq p_j$, the completion time of both jobs $i$ and $j$ are decreased. The other jobs after job $j$ can be scheduled earlier. The schedule $\Sigma(K, i)$ then is dominated.

In the case where $p_i < p_j$, the jobs between jobs $j$ and $i$ can be scheduled earlier and the jobs after $i$ in $\Sigma(K, i)$ are delayed (see Figure 2). This increases the completion time of each of these jobs at most by $p_j - p_i$. The number of these jobs is $\text{card}(N - J(K)) - 1)$. In addition, we have $C_j(S) - C_i(\Sigma(K, i)) \leq p_j - p_i$ and $C_i(S) - C_j(\Sigma(K, i)) = E_i(\Delta_j) - E_j(\Delta_j)$. From the assumption of the theorem, we have

$$F(S) - F(\Sigma(K, i)) \leq \{\text{card}(N - J(K)) - 1\}(p_j - p_i) + p_j - p_i + E_i(\Delta_j) - E_j(\Delta_j) \leq 0.$$

$\Sigma(K, i)$ then is dominated. ■

PROOF (of Theorem 10): Consider the schedule $\Sigma(K, i)$. We construct another schedule $S$ by interchanging the positions of jobs $i$ and $j$.

If $E_i(\Delta_j) \leq E_j(\Delta_j)$, taking into account the fact that $p_i \geq p_j$, we can obtain $E_i(\Delta_j) - p_i \leq E_j(\Delta_j) - p_j$. From the definition of $k$ it is easy to see that $\text{card}(J(K)) \geq k$. If $E_i(\Delta_j) > E_j(\Delta_j)$, considering the assumptions of the theorem, we have $2(p_i - p_j) \geq p_i - p_j \geq (E_i(\Delta_j) - E_j(\Delta_j)) \cdot (\text{card}(J(K)) - k + 2) \geq 2 \cdot (E_i(\Delta_j) - E_j(\Delta_j))$.

Consequently, we have in any case $R_i(\Delta_j) = E_i(\Delta_j) - p_i \leq E_j(\Delta_j) - p_j = R_j(\Delta_j)$. The jobs after $i$ in $\Sigma(K, i)$ can then be scheduled earlier. The jobs between jobs $i$ and $j$ are delayed (see Figure 3). The interchange increases the completion time of each of these jobs at most by $E_i(\Delta_j) - E_j(\Delta_j)$. The number of these jobs is $\text{card}\{J(K)\} - k$. In addition, we have $C_j(S) - C_i(\Sigma(K, i)) \leq p_j - p_i + 1$ $(E_i(\Delta_j) - E_j(\Delta_j))$ and $C_i(S) - C_j(\Sigma(K, i)) = E_i(\Delta_j) - E_j(\Delta_j)$. From the assumption of the theorem, we have

$$F(S) - F(\Sigma(K, i)) \leq \{\text{card}(J(K)) - k\}(E_i(\Delta_j) - E_j(\Delta_j)) + p_j - p_i + E_i(\Delta_j) - E_j(\Delta_j) + E_i(\Delta_j) - E_j(\Delta_j) \leq 0.$$

$\Sigma(K, i)$ then is dominated. ■

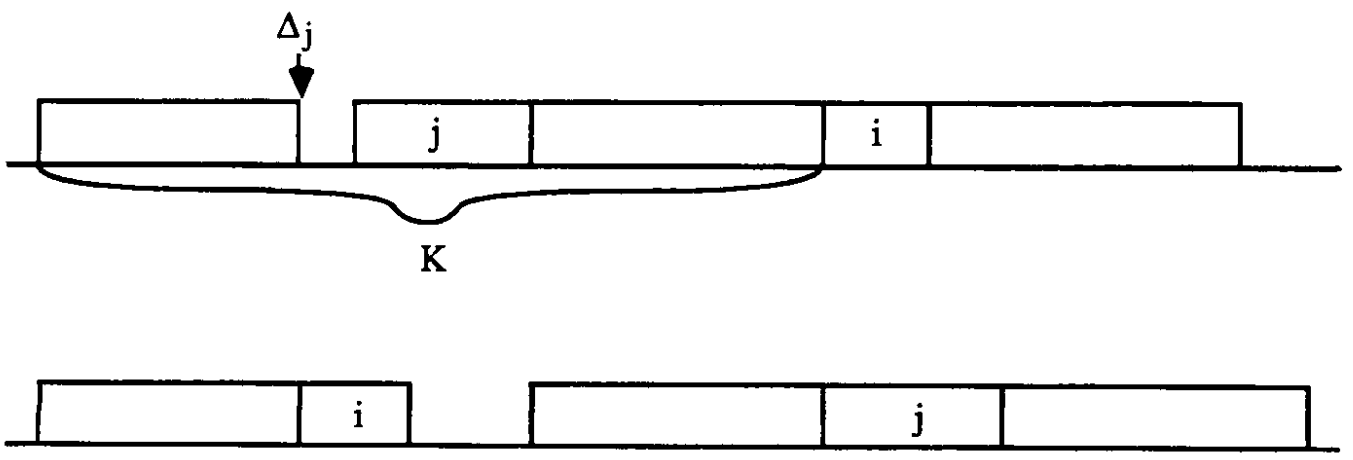


**Figure 2.** Interchange the positions of jobs $i$ and $j$. (a) Schedule $\Sigma(K, i)$. (b) Schedule $S$.

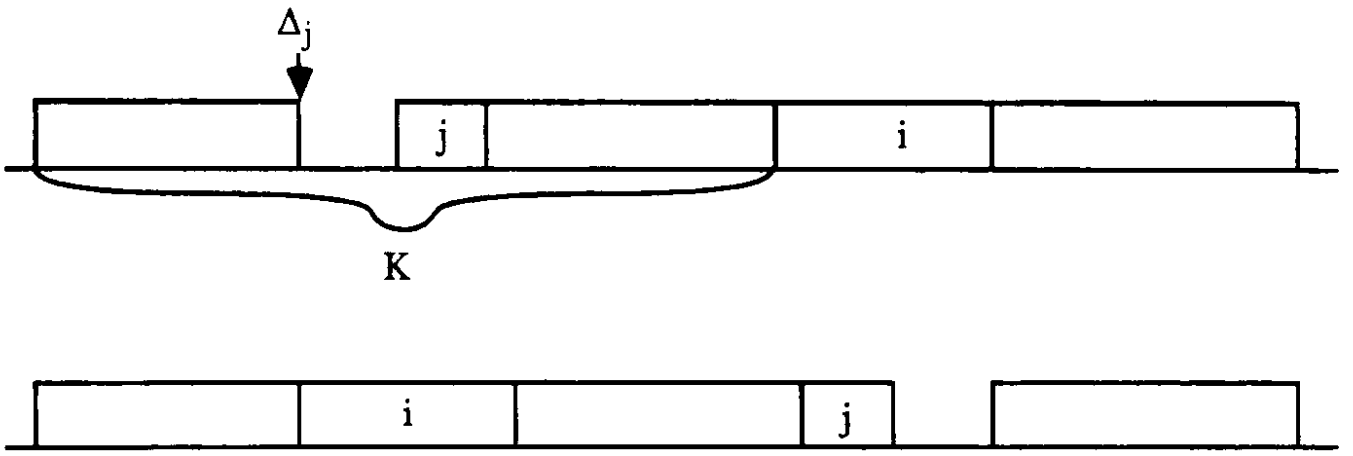


**Figure 3.** Interchange of positions of jobs $i$ and $j$. (a) Schedule $\Sigma(K, i)$. (b) Schedule $S$.

Before proving Theorem 11, we prove a lemma which will be used in the proof of Theorem 11.

LEMMA 1: Given a set $W$ of jobs, if we construct an optimal schedule $S1$ starting from the moment $t1$ and an optimal schedule $S2$ starting from the moment $t2$ ($>t1$), then $F(S2) - F(S1) \leq \text{card}(W)(t2 - t1)$.

PROOF: We construct another schedule $S$ starting from the moment $t2$ in which the jobs are scheduled in the same order as in $S1$, the completion time of each job is increased at most by $t2 - t1$. We then have $F(S) - F(S1) \leq \text{card}(W)(t2 - t1)$. In addition, from the definition of $S2$, we have $F(S2) \leq F(S)$, which gives us straightforwardly $F(S2) - F(S1) \leq \text{card}(W)(t2 - t1)$. ■

PROOF (of Theorem 11): Consider the schedule $\Sigma(K, i)$ and the schedule composed of $K'$ and the optimal partial schedule of jobs $N - J(K')$, behind $K'$.

If $R_j(\Phi(K')) \leq R_j(\Phi(K|i))$, it is obvious that $\Sigma(K, i)$ is dominated. If $R_j(\Phi(K')) > R_j(\Phi(K|i))$, according to Lemma 1 the difference of total flow time of the optimal partial schedules of $N - J(K')$ starting from the moment $R_j(\Phi(K'))$ and starting from the moment $R_j(\Phi(K|i))$ is at most $\text{card}\{N - J(K')\} \cdot \{R_j(\Phi(K')) - R_j(\Phi(K|i))\}$. From the assumptions, the theorem then is proved. ■

In the remainder of the section, we provide some properties indicating that some dominance theorems are dominated by another. That is why they are eliminated in our new branch-and-bound algorithm presented in Section 7. In [5], we proved the following property, which shows that the $F$-active schedules verify Theorem 2.

PROPERTY 1: Given a partial schedule $K$, if a job $i \in N - J(K)$ satisfies $p_i < p_k$ for all $k \in N - (J(K) \cup \{i\})$ and another job $j \in N - J(K)$ satisfies $R_j(\Phi(K)) \geq R_i(\Phi(K))$, then job $i$ precedes job $j$ in an $F$-active schedule.

This property shows that an $F$-active schedule satisfies Theorem 2 except in some cases where we have $p_i = p_j$. However, it does not really matter, as we

can avoid these cases by suitably breaking the ties. With this precaution, our dominance property using only $F$-active schedules is stronger than Theorem 2.

In fact, Theorems 3 and 4 describe a particular case of active schedules. Since the criterion "minimization of total (weighted) completion (flow) time" is a regular criterion, and the active schedule subset is dominant for regular criteria (Baker [2], Conway, Maxwell, and Miller [6]), we immediately obtain the following property.

PROPERTY 2: Theorems 3 and 4 can immediately be derived from the statement that the active subset is dominant for regular criteria.

Furthermore, it is not difficult to obtain the following property.

PROPERTY 3: Theorem 6 dominates Theorem 5.

From the comparison, among the dominance theorems presented so far, we can eliminate Theorems 2–5.

## 5. CONDITIONS FOR OPTIMALITY

In this section, we discuss some sufficient conditions for optimality. They are based on some priority rules which we must first present. In the following description, $K$ is initially set to $\emptyset$.

*The earliest completion time (ECT) rule (Dessouky and Deogun* [8]): Select job $i \in N - J(K)$ with the smallest completion time $E_i(\Phi(K)) \leq E_j(\Phi(K))$ for all $j \in N - J(K)$. Break ties by choosing $i$ with $\min(R_i(\Phi(K))$. Schedule job $i$ and update $K$.

*The smallest remaining procesing time (SRPT) rule for the problem with preemptive jobs (Chandra* [4]): When we have to select a job among those which are waiting, the one with the smallest remaining processing time is chosen. Furthermore, an arriving job preempts the job being processed if the processing time of the new arrival is strictly smaller than the remaining processing time of the job in process.

The SRPT rule is optimal for the problems with preemption (Schrage [13]). Dessouky and Deogun [8] proved the following sufficient condition for optimality.

THEOREM 12: A sufficient condition for the optimality of a sequence is that every block (a block is a set of contiguously scheduled jobs) is ordered according to ECT and starts with the job having the smallest release date.

In fact, this theorem is equivalent to the following property, which is well known and applies to all scheduling problems.

PROPERTY 4: Given a problem $\Pi$ in which the jobs are not preemptive, we construct a problem $\Pi'$ identical to $\Pi$ except that the jobs are preemptive in $\Pi'$. If there is a schedule $S'$ optimal for $\Pi'$ without preemption, then $S'$ is also optimal for problem $\Pi$.

In order to prove that Theorem 12 is equivalent with Property 4, we first prove the following lemma.

LEMMA 2: Let $\Pi$ and $\Pi'$ be problems as described in Property 4. If there is a schedule $S$ for $\Pi$ in which every block is ordered according to ECT and starts with the job having the smallest release date, then there is a schedule $S'$ for $\Pi'$ in which the jobs are ordered according to SRPT, there is no preemption, the jobs are scheduled in the same order as in $S$, and vice versa.

PROOF: We only have to prove that, if there is a schedule $S$ for $\Pi$ in which every block is ordered according to ECT and starts with the job having the smallest release date, then there is a schedule $S'$ for $\Pi'$ in which the jobs are ordered according to SRPT without preemption and the jobs are scheduled in the same order as in $S$. The converse can be proved in a similar way.

Let us consider the schedule $S$. Denote by $x1$ the first job in $S$. We have the following relation:

$$\forall i \neq x1, \qquad R_{x1}(0) \leq R_i(0) \quad \text{and} \quad E_{x1}(0) \leq E_i(0).$$

This implies that the job $x1$ in problem $\Pi'$ at the moment $R_{x1}(0)$ $(=r_{x1})$ is a job with the smallest remaining processing time. Any job available at time $\Delta_{x1}$ $(=0)$ cannot preempt the processing of $x1$. For any job $j$ arriving at time $r_j > R_{x1}(0)$, we have $E_{x1}(0) - r_j \leq E_j(0) - r_j$, which means that the remaining processing time of job $x1$ at time $r_j$ is also smaller than that of job $j$ at this moment. The processing of job $x1$ cannot be preempted.

The same reasoning is also applicable to the jobs occupying other positions in $S$ than the first one. We can therefore construct a schedule $S'$ where there is no preemption and the jobs are scheduled in the same order as in $S$. ■

From Lemma 2, Theorem 12 is similar to saying: «Given a problem $\Pi$, we construct a problem $\Pi'$ identical to $\Pi$ except that the jobs are preemptive in $\Pi'$; if there exists a schedule $S'$ for $\Pi'$ verifying SRPT, then $S'$ is an optimal schedule for $\Pi$». However, the latter statement is obvious following Property 4 and the fact that $S'$ is an optimal schedule for problem $\Pi'$. Therefore, the following property holds.

PROPERTY 5: Theorem 12 can be derived from Property 4.

Deogun [7] proved the following theorem, which can be derived from Theorem 12.

THEOREM 13: A sufficient condition for the optimality of a block is that the first job in the block has the smallest release date of any job in the block, and that all jobs in the block are sequenced according to SPT.

It is clear that for any block, if the first job in the block has the smallest release date of any job in the block, and if all jobs in the block are sequenced according to SPT, then all jobs in the block are also sequenced according to

ECT. Therefore, any schedule verifying the condition for the optimality described in Theorem 13 also verifies the condition for optimality described in Theorem 12. However, a schedule verifying the condition for optimality described in Theorem 12 does not verify with certainty the condition for optimality described in Theorem 13, as shown by the following example: $n = 2$, $r_1 = 3$, $p_1 = 4$, $r_2 = 0$, $p_2 = 5$. A schedule composed of 2 followed by 1 is optimal according to Theorem 12, but Theorem 13 fails to show the optimality, which leads to the following property.

PROPERTY 5: Theorem 12 is more general than Theorem 13.

Consequently, it is sufficient to check whether there exists a schedule satisfying SRPT without preemption.

## 6. LOWER BOUNDS

In the literature, we can find many lower bounding schemes. An obvious way to obtain a lower bound is to relax the problem to a problem with preemption and to apply the SRPT rule to the relaxed problem, since the SRPT rule provides an optimal solution to the relaxed problem. This lower bound is denoted by LB_SRPT. In this section, we review some other lower bounds used in a variety of branch-and-bound algorithms that we present in the next section.

### 6.1. Lower Bound Due to Dessouky and Deogun

Dessouky and Deogun [8] studied the same problem considered in this article, and proposed a lower bound noted LB_DD, which is computed in the following way: Given a problem $\Pi$, we construct a schedule $S$ verifying ECT. We construct another problem $\Pi'$ and a schedule $S'$ for $\Pi'$ by keeping the order of jobs in schedule $S$ and by modifying the release dates of jobs in the following way (where $[i]$ is the job occupying the $i$th position in a schedule and $C'_{[i]}$ is the completion time of whichever job occupies the $i$th position in schedule $S'$):

$C'_{[0]} = 0$;
For $i$ from 1 to $n$ do
begin

$$m_i = \min_{j \geq i}(r_{[j]});$$
$$r'_{[i]} = \min(r_{[i]}, \max(C'_{[i-1]}, m_i));$$
$$C'_{[i]} = \max(r'_{[i]}, C'_{[i-1]}) + p_{[i]};$$

end

The quantity $\text{LB\_DD} = \sum_{i=1}^{n} (C'_{[i]} - r_{[i]})$ is a lower bound of the problem $\Pi$.

### 6.2. Lower Bound Due to Bianco and Ricciardelli

Bianco and Ricciardelli [3] studied the problem of minimizing total weighted completion time and proposed a lower bound noted by LB_BR, which is com-

puted in the following way:

$$\text{LB\_BR} = \sum_{i=1}^{n} p_i + \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Delta_{i,j},$$

where

$$\Delta_{i,j} = \begin{cases} \max(r_i + p_i - r_j, 0) - \max(r_i + p_i - r_j - p_j, 0), & \text{if } r_i \le r_j, \\ \max(r_j + p_j - r_i, 0) - \max(r_j + p_j - r_i - p_i, 0), & \text{if } r_i > r_j. \end{cases}$$

### 6.3. Lower Bound Due to Deogun

Deogun [7] used a lower bound (noted LB_D) computed in the following way. The problem is relaxed to a problem with equal release dates such that the release dates of all jobs are zero. The SPT rule is then applied to the relaxed problem.

### 6.4. Lower Bounds Due to Hariri and Potts

Hariri and Potts [10] also considered the weighted version of the problem. With each job $i$ is associated a weight $w_i$ (for the problem considered in this paper, $\forall i$, $w_i = 1$). Using Lagrangian relaxation, they derived a lower bound denoted by LB_HP.

The lower bound LB_HP is based on a heuristic EST that we have to present at first. At the beginning of the scheduling, $K$ is initially set to $\emptyset$.

*The earliest start time (EST) rule*: Select job $i \in N - J(K)$ with the smallest earliest beginning time $R_i(\Phi(K)) \le R_j(\Phi(K))$ for all $j \in N - J(K)$. Break ties by choosing $i$ with $\min(p_i/w_i)$, and further ties by choosing $i$ with $\min(i)$. Schedule $i$ and update $K$.

Assume that the sequence generated by EST heuristic is $(1, 2, \ldots, n)$ and that the completion time in this schedule is $C_i^*$, where

$$C_1^* = r_1 + p_1 \quad \text{and} \quad C_i^* = \max(r_i, C_{i-1}^*) + p_i.$$

The jobs in the EST schedule are then partitioned into $k$ blocks $S_1, S_2, \ldots, S_k$, where job $v_j$ is the last job in a block $S_j$ if $C_{v_j}^* \le r_i$ for $i = v_j + 1, v_j + 2, \ldots, n$. A block $S_j$ consisting of the jobs $\{u_j, u_j + 1, \ldots, v_j\}$ is defined by the following conditions:

(1) $u_j = 1$ or job $u_j - 1$ is the last job in a block.
(2) Job $i$ is not the last job in a block for $i = u_j, u_j + 1, \ldots, v_j - 1$.
(3) Job $v_j$ is the last job in a block

The lower bound is given by

$$\text{LB\_HP} = \sum_{i=1}^{n} w_i(C_i^* - r_i) + \sum_{i=1}^{n} \lambda_i^*(r_i + p_i - C_i^*),$$

where

$$\lambda_i^* = \begin{cases} 0, & \text{if } i = u_j, \\ \max\{0, w_i + (\lambda_{i-1}^* - w_{i-1})(p_i/p_{i-1})\}, & \text{if } i = u_j + 1, u_j + 2, \ldots, v_j. \end{cases}$$

Hariri and Potts also proposed an improved lower bound denoted by ILB_HP, which is computed in the following way: We reorder within each block in non-decreasing order of $\lambda_i^*$ to give a permutation $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ such that $S_j = \{\pi(u_j), \pi(u_j + 1), \ldots, \pi(v_j)\}$ and $\lambda_{\pi(u_j)}^* \le \lambda_{\pi(u_j+1)}^* \le \cdots \le \lambda_{\pi(v_j)}^*$ for all $j$. Further conditions are the following:

$$S_j^{(0)} = S_j, \qquad \text{for } j = 1, 2, \ldots, k,$$

$$S_j^{(h)} = S_j^{(h-1)} - \{\pi(u_j + h - 1)\},$$

$$\text{for } h = 1, 2, \ldots, v_j - u_j; \qquad j = 1, 2, \ldots, k,$$

$$\mu_j^{(h)} = \lambda_{\pi(u_j+h)}^* - \lambda_{\pi(u_j+h-1)}^*,$$

$$\text{for } h = 1, 2, \ldots, v_j - u_j; \qquad j = 1, 2, \ldots, k,$$

$$b_j^{(h)} = \sum_{i \in S_j^{(h)}} (r_i + p_i), \qquad \text{for } h = 1, 2, \ldots, v_j - u_j; \qquad j = 1, 2, \ldots, k.$$

Let $\beta_j^{(h)}$ denote the sum of completion times for the jobs in $S_j^{(h)}$ when they are sequenced according to the SRPT rule, for $h = 1, 2, \ldots, v_j - u_j$; $j = 1, 2, \ldots, k$. The improved lower bound is defined as

$$\text{ILB\_HP} = \text{LB\_HP} + \sum_{k=1}^{k} \sum_{h=1}^{v_j - u_j} \mu_j^{(h)}(\beta_j^{(h)} - b_j^{(h)}).$$

Recently, Ahmadi and Bagchi [1] proved that among the lower bounds, LB_SRPT is the dominant in quality as well as in complexity.

## 7. BRANCH-AND-BOUND ALGORITHMS

### 7.1. The New Branch-and-Bound Algorithm BB_C

The proposed branch-and-bound algorithm uses the same scheme as the one used in the branch-and-bound algorithms proposed in [3] and [8]: During the computation, we keep a list of unexplored nodes arranged in increasing order according to the lower bounds of nodes, with ties broken by a nonincreasing number of scheduled jobs. Each node represents a partial schedule. The algorithms always try to develop the head of the list. Before any new node is created, a series of dominance properties are checked. For each node of the search tree, which cannot be eliminated by dominance properties, a lower bound is calculated. If the lower bound is greater than or equal to the total flow time of a known complete solution, this node is also eliminated.

In BB_C, supplementary nodes are eliminated by using Theorem 11. The dominance properties used are Corollary 1, Theorems 6, 7, 9, and 10, and the dominance of active subset. The test of optimality is realized by verifying whether, in the solution provided by SRPT rule, some preemption takes place. The lower bound used is LB_SRPT. The initial solution is given by the best solution found by APRTF and PRTF. For each node, we also compute an upper bound using the APRTF heuristic, because it seems the most efficient among the heuristics presented so far.

### 7.2. The Algorithm of Chandra, BB_RC

In the algorithm of Chandra (noted BB_RC) for the preempt-repeat model, a partitioning scheme was used, which is based on the following theorem.

THEOREM 14 (Chandra [4]): Suppose that the jobs are numbered in nondecreasing order of their release dates; if $m$ is the first $j$ such that $r_j + \sum_{i=1}^{j} p_i \leq r_{j+1}$, then the first $m$ jobs and the remaining $n - m$ jobs can be scheduled independently.

After the partitioning phase, each subproblem is solved by a branch-and-bound technique. In this technique, the choice of a node to branch is the same as in BB_C. The dominance of active schedules is used. The lower bound is provided by SRPT priority rule. The initial solution is obtained by applying the EST heuristic.

### 7.3. The Algorithm of Dessouky and Deogun, BB_DD

In BB_DD, the dominance properties used are Theorems 2, 3, and 5. The lower bound is provided by LB_DD. The initial solution is the best one given by ECT and EST. For the noneliminated node, an upper bound is computed by applying ECT.

### 7.4. The Algorithm of Bianco and Ricciardelli, BB_BR

In BB_BR, the dominance properties used are Theorems 2, 4, 6, and 7. The lower bound is provided by LB_BR. The initial solution is provided by EST or ECT.

### 7.5. The Algorithm of Deogun, BB_D

In the algorithm of Deogun (noted BB_D), a partitioning scheme was used, which is based on the following theorem.

THEOREM 15 (Deogun [7]): In a SPT sequence composed of two partial subsequences $K$ and $K'$ ($K$ preceding $K'$), if $\forall i \in J(K')$, $r_i \geq \Phi(K)$, then the initial problem can be partitioned into two independent subproblems with, respectively, the jobs of $K$ and the jobs of $K'$.

After the partitioning phase, each subproblem is tested by the optimality test (Theorem 13). If the test fails, a branch-and-bound technique is used. In this technique, the choice of a node to branch is the same as in BB_DD, BB_BR, and BB_C. The dominance properties used are Theorems 2, 3, 5, and 8. The lower bound is provided by LB_D. For each created node, an upper bound is calculated by applying the SPT rule.

### 7.6. The Algorithms of Hariri and Potts, BB_HP and IBB_HP

The branch-and-bound algorithm of Hariri and Potts (noted BB_HP) differs by the choice of the node to branch. It chooses a node which has the smallest lower bound among nodes in the most recently created subset. The dominance properties used are Theorems 2 and 3. The lower bound used is LB_HP or ILB_HP (in this case, the corresponding algorithm is denoted by IBB_HP).

### 7.7. Remarks

It seemed interesting to incorporate the partitioning schemes proposed respectively by Chandra and Deogun and Theorem 8 in our new branch-and-bound algorithm BB_C. But in our experiment, the partitioning schemes failed to partition problems into subproblems. Even if it happened, the problems were partitioned into two subproblems, the one having a very small size and the other having a very great size. They are thus of no interest. This indicates that the generated problems are rather difficult: It is not easy to decompose them into subproblems.

Concerning Theorem 8, on one hand, it could not be implemented efficiently with our data structure (important increase of computation time), and on the other hand, it restricted the search very slightly. Probably it is often dominated by other dominance properties.

## 8. COMPUTATIONAL RESULTS

In this section, we give some numerical results in order to evaluate the performances of the proposed branch-and-bound algorithm, and compare it with other existing branch-and-bound algorithms. All the branch-and-bound algorithms were programmed and tested on a SUN 4 SPARC machine using a C compiler. We randomly generated nine samples of examples with, respectively, 20, 30, 40, 50, 60, 70, 80, 90, and 100 jobs. The examples were generated in the same way as in Hariri and Potts [10]. The processing times were generated

**Table 2.** Mean number of nodes considered.

| $n$ | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
|---|---|---|---|---|---|---|---|---|---|
| BB_RC | 210.68 | 633.18 | 31014.3 | — | — | — | — | — | — |
| BB_DD | 557.54 | — | — | — | — | — | — | — | — |
| BB_HP | 64.24 | 579.34 | 5663.20 | 50781.8 | — | — | — | — | — |
| IBB_HP | 53.60 | 409.34 | 3748.16 | — | — | — | — | — | — |
| BB_C | 19.68 | 57.16 | 155.62 | 279.54 | 963.98 | 1870.32 | 2490.70 | 5800.32 | 14057.3 |

**Table 3.** Mean number of nodes generated.

| $n$ | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
|---|---|---|---|---|---|---|---|---|---|
| BB_RC | 313.02 | 639.50 | 31028.1 | — | — | — | — | — | — |
| BB_DD | 581.48 | — | — | — | — | — | — | — | — |
| BB_HP | 79.90 | 610.10 | 5723.22 | 50880.4 | — | — | — | — | — |
| IBB_HP | 57.20 | 416.90 | 3767.78 | — | — | — | — | — | — |
| BB_C | 19.96 | 57.92 | 158.34 | 288.22 | 983.50 | 1903.60 | 2545.02 | 5929.72 | 14202.8 |

between 1 and 100. The release dates were generated between 0 and $50.5 \cdot n \cdot \lambda$. Ten values of $\lambda$ (0.2, 0.4, 0.6, 0.8, 1.0, 1.25, 1.50, 1.75, 2.0, 3.0) permitted us to generate 50 examples for each sample (five examples for each value of $\lambda$).

We applied the existing branch-and-bound algorithms, BB_BR, BB_RC, BB_D, BB_DD, BB_HP, and IBB_HP and the new branch-and-bound algorithm BB_C to these samples. We give a limit of 15 hours (from 6 p.m. to 9 a.m. the next day) for each algorithm to solve all the examples of the sample. We found that BB_BR and BB_D could solve none of the samples. BB_DD could solve only the sample of 20 jobs. BB_RC could solve problems with up to 40 jobs. BB_HP could solve problems with up to 50 jobs, but IBB_HP failed to solve the sample with 50 jobs. This is because the computation of an improved lower bound ILB_HP is much more expensive than the computation of LB_HP.

Tables 2–5 summarize, respectively, the mean number of nodes considered, mean number of nodes generated, mean maximal number of nodes in the list which measures the memory core used, and computation time for each branch-and-bound algorithms and for each sample. A considered node is a node from which the algorithm tried to create descendant nodes, while a generated node is a node that failed to be eliminated.

From the tables, we can immediately find the superiority of the proposed branch-and-bound algorithm to existing ones, whatever the performance measure. It can solve problems with up to 100 jobs within a reasonable computation time, 50 jobs more than the largest size of examples solved by existing algorithms. It also is true that for each example, the new branch-and-bound algorithm dominates for any performance measure.

Looking at the evolution of the computation time versus the number of jobs, we can find that it is doubled or tripled when the number of jobs is increased by 10. This is consistent with the NP-hardness of the problem considered in this article.

The examples for which the computation time counter was overflowed essentially were generated with $\lambda$ between 0.4 and 1.0. This is not surprising at all.

**Table 4.** Mean maximal number of nodes in the list.

| $n$ | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
|---|---|---|---|---|---|---|---|---|---|
| BB_RC | 7.44 | 17.10 | 27.5 | — | — | — | — | — | — |
| BB_DD | 136.36 | — | — | — | — | — | — | — | — |
| BB_HP | 23.14 | 46.64 | 83.00 | 132.2 | — | — | — | — | — |
| IBB_HP | 11.20 | 22.92 | 47.20 | — | — | — | — | — | — |
| BC_C | 4.54 | 9.26 | 22.90 | 41.70 | 134.04 | 170.52 | 264.14 | 656.32 | 1343.26 |

**Table 5.** Mean computation time (in CPU ms).

| $n$ | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
|---|---|---|---|---|---|---|---|---|---|
| BB_RC | 307 | 1609 | 123 711 | — | — | — | — | — | — |
| BB_DD | 1282 | — | — | — | — | — | — | — | — |
| BB_HP | 141 | 1986 | 32 262 | 343 075(2) | — | — | — | — | — |
| IBB_HP | 1938 | 39 298 | 526 819(8) | — | — | — | — | — | — |
| BB_C | 51 | 330 | 1592 | 5304 | 23 259 | 64 641(1) | 92 465 | 201 041 | 392 717(4) |

*Note*: The number in the parentheses is the number of examples for which the size of the CPU counter (32 bits) was overflowed. In this case, we give the lower bound of the real computation time by replacing the computation times of these examples with the counter capacity (2 147 484 ms)

The problem is relatively simple when $\lambda$ is small, because the release dates are not scattered. It becomes quickly equivalent to a problem with equal release dates after scheduling a few jobs and can be solved by the SPT rule, which is a polynomial algorithm. When $\lambda$ is large, the release dates are so scattered that there are a small number of active schedules. Since it is sufficient to consider active schedules, the search of an optimal solution among active schedules becomes easy.

The branch-and-bound algorithm is better than the previously published algorithms due to two types of improvements: the new dominance properties and the efficient heuristics. It would be interesting to know which type of improvement is the most significant. For this, we have transformed the algorithm by omitting all the new dominance properties (Theorems 9–11). We call this new algorithm by BB_C − D (D stands for dominance properties). We have also constructed an algorithm by replacing, for the initial solution and for all the computed upper bounds, our heuristics by EST, which seems the best among previously published heuristics. This algorithm is called BB_C − H (H stands for heuristics). We finally constructed another algorithm by omitting all the improvements of this article. It is called BB_C − A (A stands for all). We tested the three algorithms on the sample with 50 jobs. The performance measures are summarized in Table 6. In BB_C − D and BB_C − A, the search tree was so large that they could not solve all the problems within 15 hours. In order to obtain some results, we have to change the choice of a node to branch for these algorithms. We decided to choose the node with the smallest lower bound among the ones created the most recently.

Table 6 indicates that the new dominance properties are very useful. The improvement due to the heuristics is rather slight. Since the heuristic APRTF is a little more complex than EST, the computation time of BB_C even is slightly greater than BB_C − H. But this is not true when $n$ is large, $n > 60$, for example, because we also tested BB_C − H on these samples.

**Table 6.** Performance measures.

| Algorithms | Mean number of nodes considered | Mean number of nodes generated | Mean maximal number of nodes in the list | Mean computation time |
|---|---|---|---|---|
| BB_C | 279.54 | 288.22 | 41.70 | 5304 |
| BB_C − H | 303.16 | 313.52 | 43.02 | 4796 |
| BB_C − D | 27 528.08 | 27 532.58 | 32.50 | 225 398 |
| BB_C − A | 27 532.40 | 27 539.12 | 34.68 | 218 418 |

## 9. CONCLUSION

We presented a necessary and sufficient condition that can be considered as a priority rule and a new dominant subset, called *F*-active subset defined on the basis on this condition. This reduced the subset of schedules to be considered and is useful for enumerative optimal algorithms to prune the search tree. We also presented some good heuristic algorithms which construct an *F*-active schedule using the new priority rule.

We have also proved some new dominance properties, which were proved to be very efficient to prune the search tree. We reviewed and discussed results found in the literature. Using some results of the literature, the *F*-active subset, the newly proved dominance properties, and also the proposed heuristics, we succeeded in conceiving a branch-and-bound algorithm which is able to solve problems with up to 100 jobs, and we compared our new branch-and-bound algorithm with all the previous ones. Our algorithm seems to be much more efficient.

## REFERENCES

[1] Ahmadi, R.H., and Bagchi U., "Lower Bounds for Single-Machine Scheduling Problems," *Naval Research Logistics*, **37**, 967–979 (1990).
[2] Baker, K.R., *Introduction to Sequencing and Scheduling*, Wiley, New York, 1974.
[3] Bianco, L., and Ricciardelli, S., "Scheduling of a Single Machine to Minimize Total Weighted Completion Time Subject to Release Dates," *Naval Research Logistics Quarterly*, **29**, 151–167 (1982).
[4] Chandra, R., "On $n/1/F$ Dynamic Deterministic Systems," *Naval Research Logistics Quarterly*, **26**, 537–544 (1979).
[5] Chu, C., "One-Machine Scheduling for Minimizing Total Flow Time with Release Dates," *Proc. of Rensselaer's Sec. Conf. on C.I.M.*, Troy, NY, May 21–23, 1990, pp. 570–576.
[6] Conway, R.W., Maxwell, W.C., and Miller, L.W., *Theory of Scheduling*, Addison-Wesley, Reading, MA, 1967.
[7] Deogun, J.S., "On Scheduling with Ready Times to Minimize Mean Flow Time," *Computer Journal*, **26**, 320–328 (1983).
[8] Dessouky, M.I., and Deogun, J.S., "Sequencing Jobs with Unequal Ready Times to Minimize Mean Flow Time," *SIAM Journal of Computing*, **10**, 192–202 (1981).
[9] Gupta, S.K., and Kyparisis, J., "Single Machine Scheduling Research," *OMEGA International Journal of Management Science*, **15**, 207–227 (1987).
[10] Hariri, A.M.A., and Potts, C.N., "An Algorithm for Single Machine Sequencing with Release Dates to Minimize Total Weighted Completion Time," *Discrete Applied Mathematics*, **5**, 99–109 (1983).
[11] Lenstra, J.K., Rinnooy Kan, A.H.G., and Brucker, P., "Complexity of Machine Scheduling Problems," *Annals of Discrete Mathematics*, **4**, 281–300 (1977).
[12] Rinnooy Kan, A.H.G., *Machine Sequencing Problems: Classification, Complexity and Computation*, Nijhoff, The Hague, 1976.
[13] Schrage, L., "A Proof of the Shortest Remaining Processing Time Discipline," *Operations Research*, **16**, 687–690 (1968).
[14] Smith, W. E., "Various Optimizers for Single Stage Production," *Naval Research Logistics Quarterly*, **3**, 59–66 (1956).

Manuscript received April, 1991
Revised manuscript received November, 1991
Accepted February 11, 1992